Git & GitHub Detailed Notes and Important Questions

1. Introduction to Git and GitHub

• Git is a distributed version control system used to track changes in source code.

• GitHub is a cloud-based platform used to host Git repositories and collaborate with other developers.

• Git allows developers to maintain project history and work on projects efficiently.

2. Why Use Git?

• Tracks every change made in the project.

• Allows developers to revert to previous versions.

• Supports teamwork and collaboration.

• Provides branching and merging features.

3. Installing and Configuring Git

• Install Git from the official website.

• Configure username using: git config --global user.name 'Your Name'.

• Configure email using: git config --global user.email 'your@email.com'.

• Check configuration using: git config --list.

4. Important Git Commands

• git init – Initialize a new repository.

• git status – Display the current state of the repository.

• git add – Add files to the staging area.

• git commit -m 'message' – Save changes permanently.

• git log – View commit history.

• git diff – Show differences between versions.

5. Working with GitHub

• Create a repository on GitHub.

• Connect local repository using git remote add origin URL.

• Push code using git push origin main.

• Download code using git clone URL.

6. Branching in Git

• A branch is an independent line of development.

• Create a branch using git branch branchName.

• Switch branch using git checkout branchName.

• Create and switch using git checkout -b branchName.

• List all branches using git branch.

7. Merging Branches

• Merge combines changes from one branch into another.

• Use git merge branchName to merge branches.

• Conflicts occur when the same lines are modified in different branches.

• Resolve conflicts manually and commit the changes.

8. Collaboration using GitHub

• Fork a repository to create your own copy.

• Clone the forked repository to your local machine.

• Make changes and push them to your repository.

• Create a Pull Request (PR) to contribute changes.

9. Pull Requests

• A Pull Request is a request to merge changes into another repository.

• It allows code review before merging.

• PRs improve collaboration and maintain code quality.

10. Git Fetch vs Git Pull

• git fetch downloads changes but does not merge them.

• git pull downloads and automatically merges changes.

• git pull is equivalent to git fetch + git merge.

11. Undoing Changes

• git restore restores modified files.

• git reset removes commits from history.

• git revert creates a new commit that undoes previous changes.

12. .gitignore File

• The .gitignore file specifies files that Git should ignore.

• Examples include node_modules, environment files, and temporary files.

13. Git Workflow

• Working Directory → Staging Area → Repository → GitHub.

• This is the standard workflow followed in most software projects.

14. Advantages of Git and GitHub

• Version control and project history.

• Easy collaboration among developers.

• Backup of code on the cloud.

• Supports open-source contributions.

• Efficient project management.

Most Important Exam Questions

1. What is Git? Differentiate between Git and GitHub.
2. What are the advantages of using Git?
3. Explain the Git workflow.
4. Explain the commands: git init, git add, git commit, and git status.
5. What is a branch in Git? Explain branching and merging.
6. What is a merge conflict and how is it resolved?
7. Differentiate between git fetch and git pull.
8. What is a Pull Request?
9. Explain collaboration using GitHub.
10. What is the purpose of the .gitignore file?
11. Differentiate between git reset and git revert.
12. Explain the complete process of pushing code to GitHub.